编译原理

Compiler Principles and Techniques

编译原理

- 主讲： 刘学博
- E-mail：
  liuxuebo@hit.edu.cn
- 32理论课时
- 16实验课时



为什么要学习编译原理

- 编程语言(100多种)

  Java, C, C++, C#, Basic, Pascal, VB, Python,

  Go, FORTRAN, JavaScript, Ruby, MASM,…

- 编译原理不是学习编程语言
  - 如何从底层去理解和设计一门编程语言

课程性质与特点

课程性质

- 是一门技术基础课

基础知识要求

- 高级程序设计语言，数据结构与算法，形式语言与自动机

主要特点

- 既有理论，又有实践
- 面向系统设计
- 涉及程序的自动生成技术

教学目的——《编译原理》是一门非常好的课程

- Alfred V.Aho：编写编译器的原理和技术具有十分普遍的意义，以至于在每个计算机科学家的研究生涯中，本课程中的原理和技术都会反复用到
- 本课程将兼顾语言的描述方法、设计与应用 (形式化)
  - 能形式化就能自动化（抽象→符号化→自动化）
  - 对程序设计语言具有更加深刻的理解
  - 体验实现自动计算的乐趣
- 涉及的是一个比较适当的抽象层面上的数据变换（既抽象又实际，既有理论又有实践）
  - 信息表示
  - 数据变换
- 一个相当规模的系统的设计
  - 总体结构
  - 若干具体的表示和变换算法

教学目的——《编译原理》是一门非常好的课程

◘玩《黑神话悟空》遭遇九九八十一难：第一难解压 第二难着色器编译

在解压完成后也不要着急，并不能第一时间进入游戏，游戏开始前还需要进行着色器编译，又需要等待一段时间，这对于急于想玩游戏的各位天命人而言又是漫长的等待，因此也被人们戏称为“第二难”。在全部准备活动结束后，才能真正进入游戏，开启全新的冒险之旅。



教学目的（续）

- 在系统级上认识算法、系统的设计
  - 具有把握系统的能力
  - 局部最优 vs. 全局最优 (木桶效益)
  - “自顶向下”和“自底向上”的系统设计方法
    - 对其思想、方法、实现的全方位讨论
- 进一步培养“计算思维能力”
  - 深入理解软件系统的非物理性质
  - 培养抽象思维能力和逻辑思维能力
  - 训练对复杂数据结构的设计和操纵能力

教学目的（续）

- 计算机专业最为恰当、有效的知识载体之一
- 综合运用下列课程所学知识
  - 高级程序设计语言
  - 汇编语言
  - 集合论与图论
  - 数据结构与算法
  - 计算机组成原理
  - 算法设计与分析
  - 形式语言与自动机

教学要求——课程要求

知识要求

- 掌握编译程序的总体结构、编译程序各个组成部分的任务、编译过程各个阶段的工作原理、编译过程各个阶段所要解决的问题及其采用的方法和技术

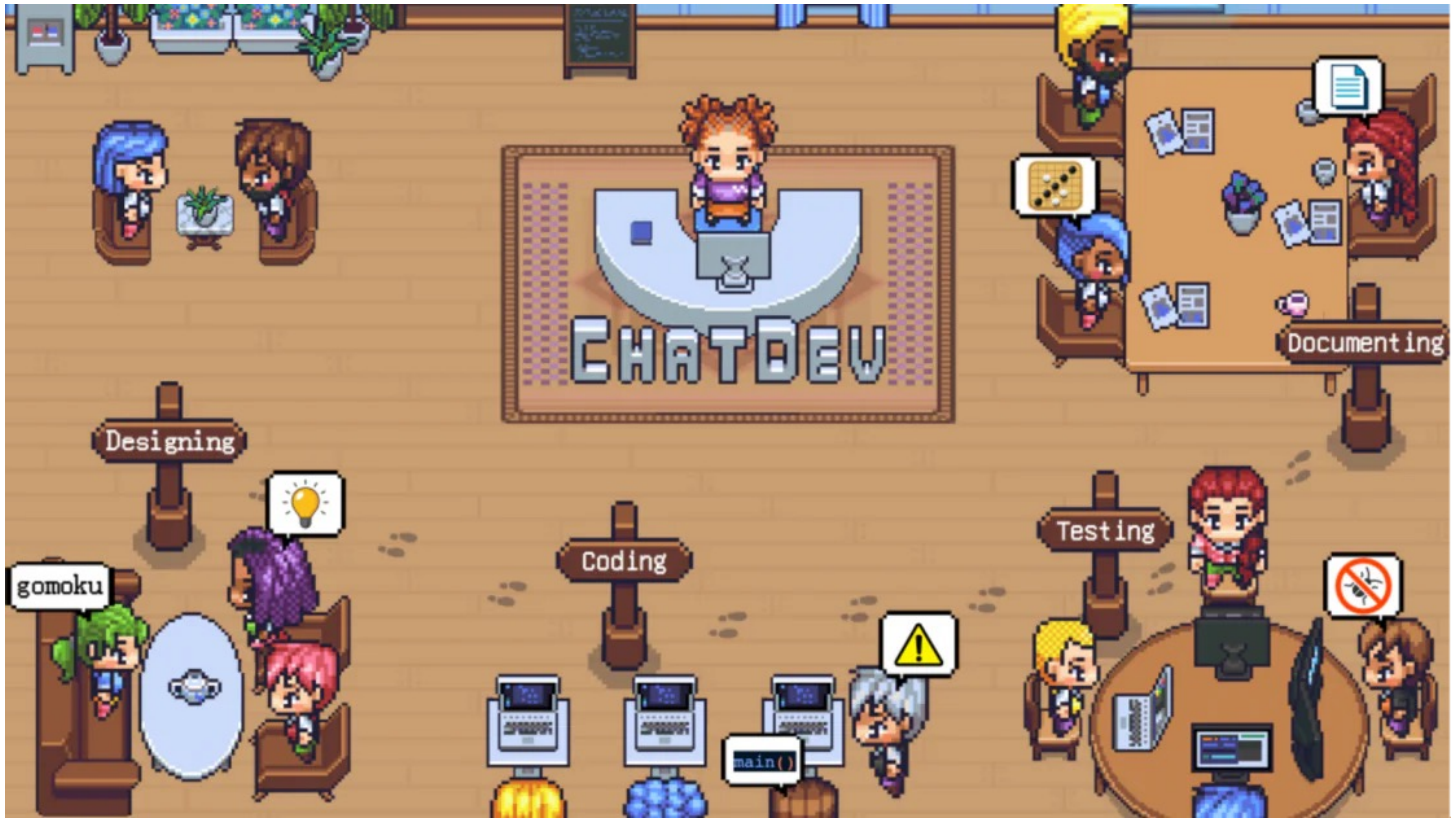
能力要求

1. 掌握程序变换基本概念、问题描述和处理方法
2. 增强理论结合实际能力
3. 修养“问题→形式化描述→计算机化” 的问题求解过程
4. 在系统级上认识算法和系统的设计，培养系统能力

为什么大模型时代仍要学习编译原理

多智能体系统不是“想到就做”，而是“描述-分解-校验-执行-优化”的工程链路

- 一个 Agent 的输出，往往会变成下一个 Agent 的输入；局部错误、歧义表达或工具调用失败，都可能沿链路被不断放大。



从多智能体系统看编译原理的现实意义

今天学编译原理，不只是为了写编译器，更是为了设计可靠的智能系统。

大模型 / 多智能体系统中的问题	编译原理给出的思想与方法
自由文本容易歧义，接口不清晰	用文法、数据结构和中间表示，把输入变成可解析、可传递、可组合的结构化对象。
一个环节出错，后续 Agent 容易级联失效	通过分阶段分析与静态检查，把错误尽量前移；不是等“跑坏了”再回头找原因。
工具调用与协同执行成本高	先做计划，再做重写与优化，减少冗余推理、重复调用和无效通信。
系统复杂后难以调试和维护	强调模块化、可追踪、可定位、可恢复，让智能系统像编译器一样具备工程可控性。

编译原理教我们的，是如何把“可能正确”的系统，做成“稳定正确”的系统。

教学要求——实验要求

实验形式

- 分析、设计、编写、调试、测试程序
- 撰写实验报告

实验内容

- 词法分析器的设计与实现　　2学时
- 语法分析器的设计与实现　　8学时
- 语义分析与中间代码生成　　4学时
- 目标代码生成　　2学时

最终成绩

- 平时成绩（10%）
  - 作业、表现、出勤等等
- 实验成绩（20%）
- 期末考试（70%）

主要内容

1. 引论
2. 高级语言及其文法
3. 词法分析
4. 自顶向下的语法分析
5. 自底向上的语法分析
6. 语法制导翻译与属性文法
7. 语义分析与中间代码生成
8. 符号表管理
9. 运行时的存储组织
10. 代码优化
11. 代码生成

教材

1. 蒋宗礼，
2. Alfred A
   北京：人
3. Alfred
   Tools(Se
   Educatic



0年2月

and Tools,
2002.2.

ques, and
, Pearson

第一章 引言

- 重点：教学目的、教学要求，课程的基本内容，编译系统的结构，编译程序的生成。
- 难点：编译程序的生成。

第一章 引言

1.1 程序设计语言

1.2 程序设计语言的翻译

1.3 编译程序的总体结构

1.4 编译程序的组织

1.5 编译程序的生成

1.6 本章小结

1.1 和

- 机
  - Lan
    - •
- 高
  - •
- 命
  - •
  - 如UNIX上的shell

```
assume cs:code, ds:data
data segment
        dw 1234h,5678h
data ends
code segment
start:  mov ax, data
        mov ds, ax
        mov ax, ds:[0]
        mov bx, ds:[2]
        mov cx, 0
        add cx, ax
        add cx, bx
        mov cx, ds:[4]
        mov ax, 4c00h
        int 21h
code ends
end start
```

```
int main
{
        int a,b,c;
        a=1234h;
        b=5678h;
        c=a+b;
        return 0;
}
```

```
2B15)
0000)
00
000 (
011 1001 0000 0000 0000 0000 (B90000)
000 0011 1100 1000 (03C8)
000 0011 1100 1011 (03CB)
000 1011 0000 1110 0000 0100 0000
000 (8B0E0400)
011 1000 0000 0000 0100 1100 (B8004C)
100 1101 0010 0001 (CD21)
```

程序设计语言的分类

- 强制式（命令式）语言(Imperative Language)
  - 通过指明一系列可执行的运算及运算的次序来描述计算过程的语言；
  - FORTRAN(段结构)、BASIC、Pascal(嵌套结构)、C……
  - 程序的层次性和抽象性不高

程序设计语言的分类

- 申述式语言（Declarative Language）
  - 着重描述要处理什么，而非如何处理的非命令式语言
  - 函数(应用)式语言(Functional Language)
    - 基本运算单位是函数，如LISP、ML……
  - 逻辑式(基于规则)语言(Logical Language)
    - 基本运算单位是谓词，如Prolog，Yacc……
  - 并发式语言(Concurrent Language)
    - 着重于如何描述潜在的并行机制，如ErLang，Fortran+MPI……

程序设计语言的分类

- 面向对象语言(Object-Oriented Language)
  - 以对象为核心，如Smalltalk、C++ 、Java、Ada(程序包)……
  - 具有识认性（对象）、类别性（类）、多态性和继承性

1.2 程序设计语言的翻译

◘ 翻译程序(Translator)

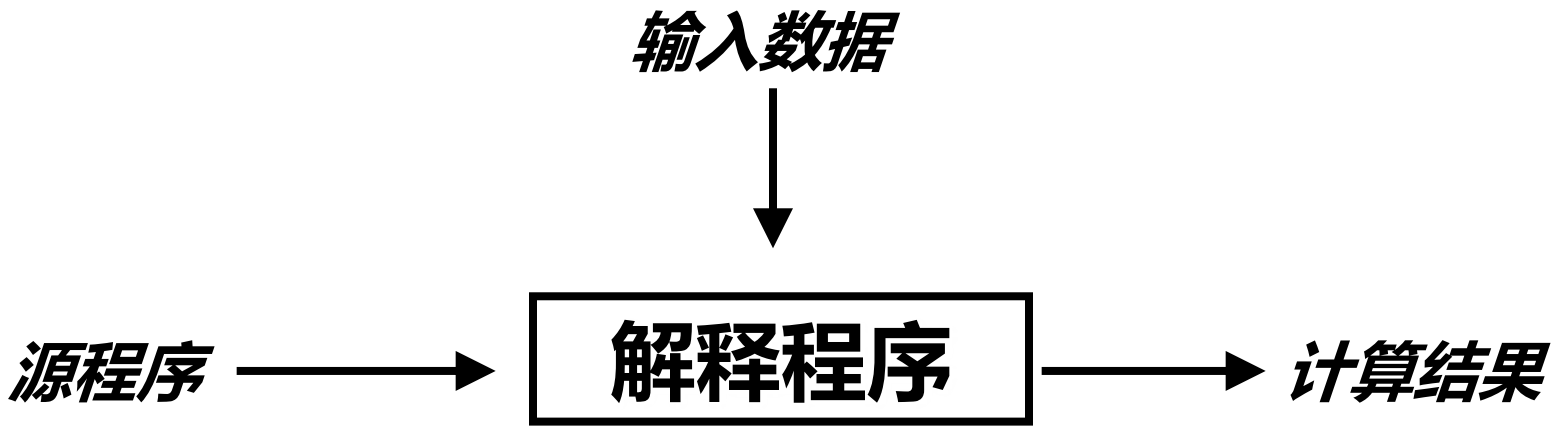
将某一种语言描述的程序(源程序——Source Program)翻译成等价的另一种语言描述的程序(目标程序——Object Program)的程序。



1.2 程序设计语言的翻译

解释程序(Interpreter)

- 一边解释一边执行的翻译程序
- 口译与笔译（单句提交与整篇提交）



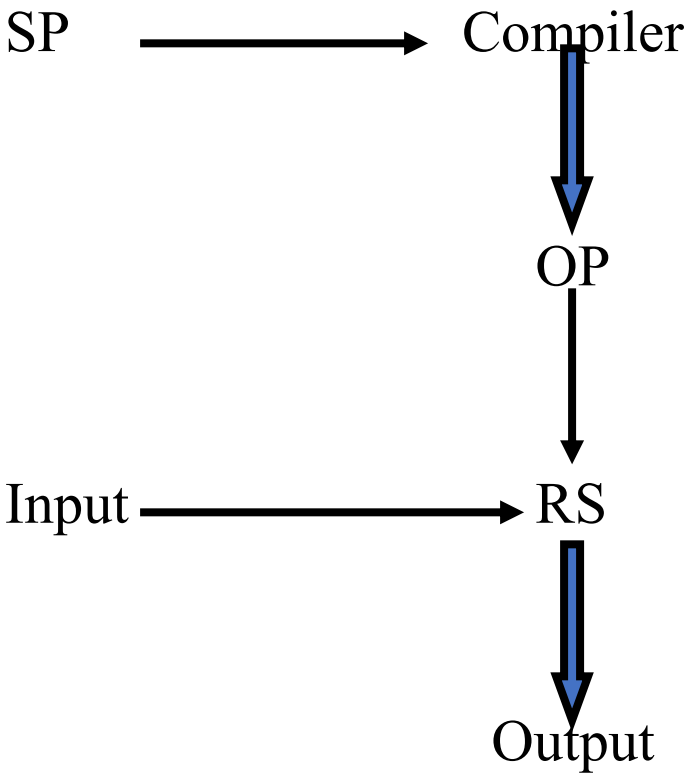
1.2 程序设计语言的翻译

编译程序(Compiler)

- 将源程序完整地转换成机器语言程序或汇编语言程序，然后再处理、执行的翻译程序
- 高级语言程序→汇编/机器语言程序



1.2 程序设计语言的翻译



S-Source

O-Object

P-Program

RS-Run Sys. **支撑环境、运行库等**

- **编译系统(Compiling System)**
  - **编译系统=编译程序+运行系统**

1.2 程序设计语言的翻译

- 其它翻译程序：
  - 汇编程序(Assembler)
  - 交叉汇编程序(Cross Assembler)
  - 反汇编程序（Disassembler)
  - 交叉编译程序（Cross Compiler)
  - 反编译程序（Decompiler）
  - 可变目标编译程序（Retargetable Compiler)
  - 并行编译程序（Parallelizing Compiler）
  - 诊断编译程序（Diagnostic Compiler)
  - 优化编译程序（Optimizing Compiler)

1.2 程序设计语言的翻译—汇总

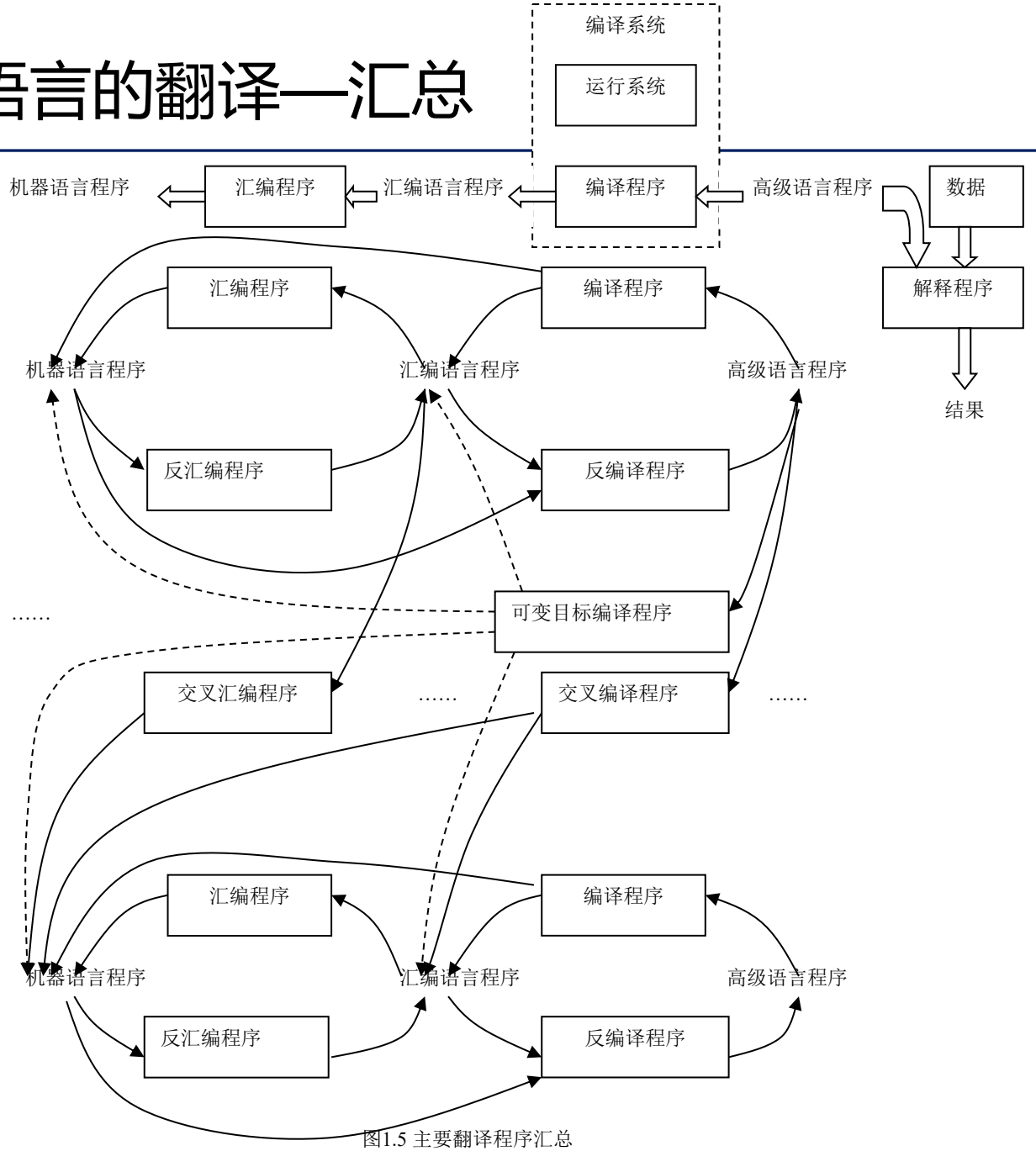
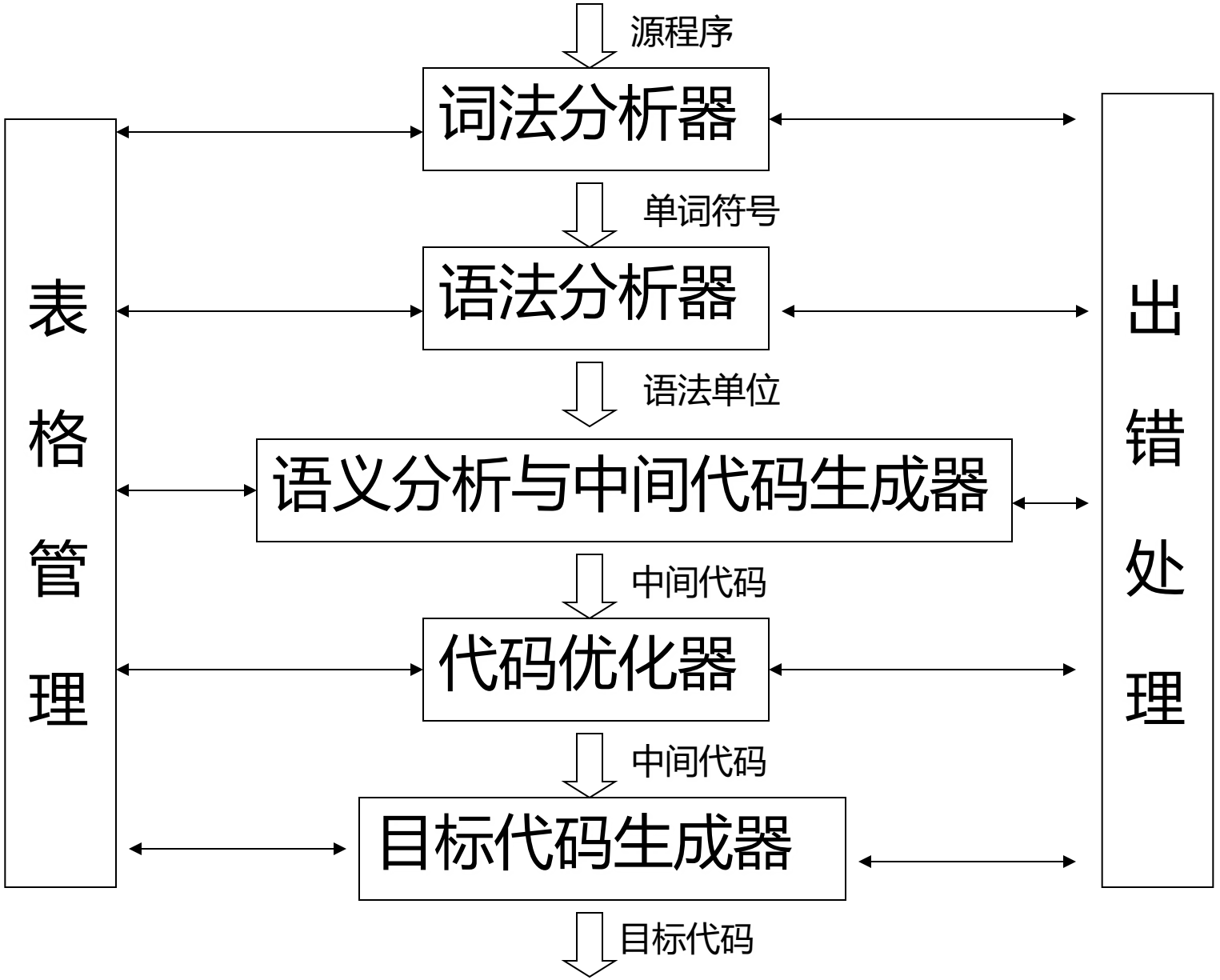


图1.5 主要翻译程序汇总

1.3 编译程序总体结构


源程序
词法分析器
单词符号
语法分析器
语法单位
语义分析与中间代码生成器
中间代码
代码优化器
中间代码
目标代码生成器
目标代码
表格管理
出错处理

(1) 词法分析

- 例：

sum=(10+20)*(num+square);

结果

- (标识符，sum)
- (赋值号，=)
- (左括号，( )
- (整常数，10)
- (加号，+)
- (整常数，20)
- (右括号，))
- (乘号，*)
- (左括号，( )
- (标识符，num)
- (加号，+)
- (标识符，square)
- (右括号，))
- (分号，;)

(1) 词法分析

- 词法分析由词法分析器(Lexical Analyzer)完成，词法分析器又称为扫描器(Scanner)

- 词法分析器从左到右扫描组成源程序的字符串，并将其转换成单词(记号—token)串；同时要：查词法错误，进行标识符登记——符号表管理。

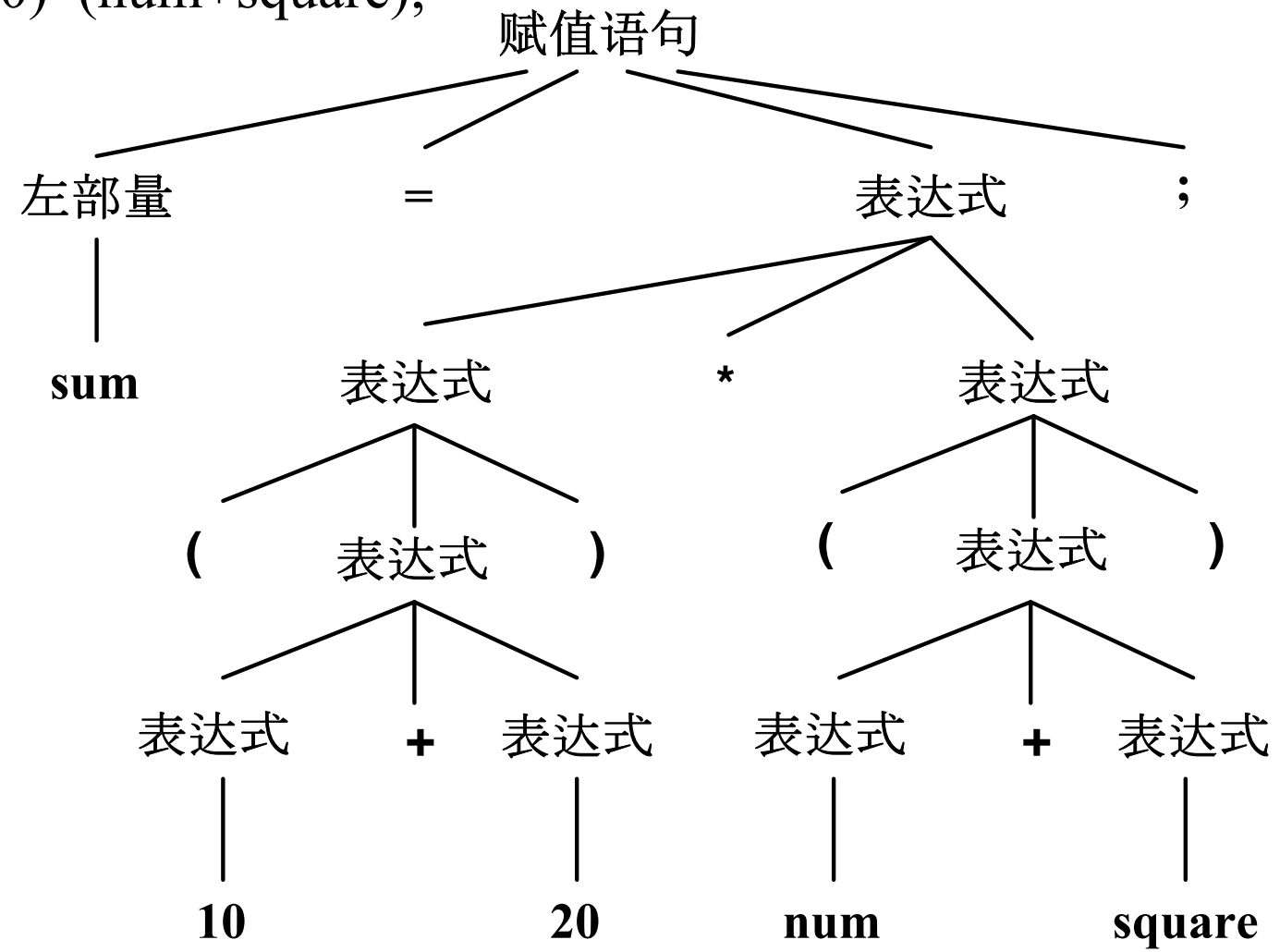
- 输入：字符串
- 输出：(种别码，属性值)——序对
  - 属性值——token的机内表示

（2）语法分析

- 语法分析由语法分析器(Syntax Analyzer)完成，语法分析器又叫Parser。
- 功能：
  - Parser实现“组词成句”
    - 将词组成各类语法成分：表达式、因子、项，语句，子程序...
  - 构造分析树
  - 指出语法错误
  - 指导翻译
- 输入：token序列
- 输出：语法成分

（2）语法分析

sum=(10+20)*(num+square);



（3）语义分析

- 语义分析(semantic analysis)一般和语法分析同时进行，称为语法制导翻译(syntax-directed translation)

- 功能：分析由语法分析器识别出来的语法成分的语义
  - 获取标识符的属性：类型、作用域等
  - 语义检查：运算的合法性、取值范围等
  - 子程序的静态绑定：代码的相对地址
  - 变量的静态绑定：数据的相对地址

（4）中间代码生成

- 中间代码 (Intermediate Code)
- 例：sum=(10+20)*(num+square);

后缀表示(逆波兰Anti- Polish Notation)

sum 10 20 + num square +*=

前缀表示(波兰Polish Notation)

= sum *+10 20+num square

四元式表示

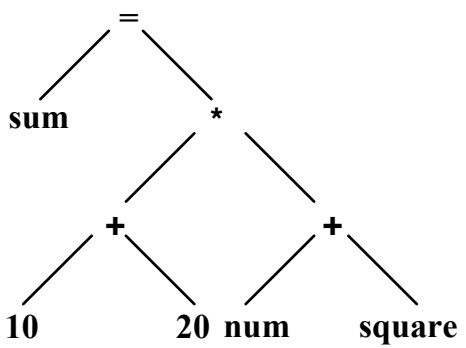
(三地址码)

(+，10，20，T_1)
(+，num，square，T_2)
(*，T_1，T_2，T_3)
(=，T_3，，sum)

三元式表示

(+，10，20)
(+，num，square)
(*，(1)，(2))
(=，sum，(3))

语法树



（4）中间代码生成

- 波兰表示问题——Lukasiewicz 1929年发明
  - 中缀表示(Infix notation):(a+①b)*(-c+②d)+③e/f
  - 波兰表示（Polish / Prefix / Parenthesis-free / Lukasiewicz notation）——也就是前缀表示
    - +③*+①a b+②@c d/ef
  - 逆波兰表示(Reverse Polish / Suffix / Postfix notation) ——也就是后缀表示
    - a b +①c@ d +②*ef/+ ③ 运算顺序从左向右

（4） 中间代码生成

中间代码的特点

- 简单规范
- 与机器无关
- 易于优化与转换

三地址码的另一种表示形式：

- T1=10+20
- T2=num+square
- T3=T1*T2
- sum=T3

其它类型的语句

例：printf(“hello”)

x := s （赋值）

param x（参数）

call f（函数调用）

注释

s 是 hello 的地址

f 是函数 printf 的地址

（5）代码优化

- 代码优化(optimization)是指对中间代码进行优化处理，使程序运行能够尽量节省存储空间，更有效地利用机器资源，使得程序的运行速度更快，效率更高。当然这种优化变换必须是等价的。
  - 与机器无关的优化
  - 与机器有关的优化

（5）代码优化——与机器无关的优化

- 局部优化
  - 常量合并：常数运算在编译期间完成，如8+9*4
  - 公共子表达式的提取：在基本块内进行的
- 循环优化
  - 强度削减
    - 用较快的操作代替较慢的操作: $X^2 \rightarrow X*X$
  - 代码外提
    - 将循环不变计算移出循环

（5）代码优化——与机器有关的优化

- 寄存器的利用
  - 将常用量放入寄存器，以减少访问内存的次数
- 体系结构
  - MIMD、SIMD、SPMD、向量机、流水机
- 存储策略
  - 根据算法访存的要求安排：Cache、并行存储体系——减少访问冲突
- 任务划分
  - 按运行的算法及体系结构，划分子任务(MPMD)

（6）目标代码生成

- 将中间代码转换成目标机上的机器指令代码或汇编代码
  - 确定源语言的各种语法成分的目标代码结构（机器指令组/汇编语句组）
  - 制定从中间代码到目标代码的翻译策略或算法
- 目标代码的形式
  - 具有绝对地址的机器指令
  - 汇编语言形式的目标程序
  - 模块结构的机器指令（需要链接程序）

(7) 表格管理

- 管理各种符号表(常数、标号、变量、过程、结构……)，查、填（登记、查找）

  源程序中出现的符号和编译程序生成的符号，为编译的各个阶段提供信息。

  - 辅助语法检查、语义检查
  - 完成静态绑定、管理编译过程

- Hash表、链表等各种表的查、填技术
- “数据结构与算法”课程的应用

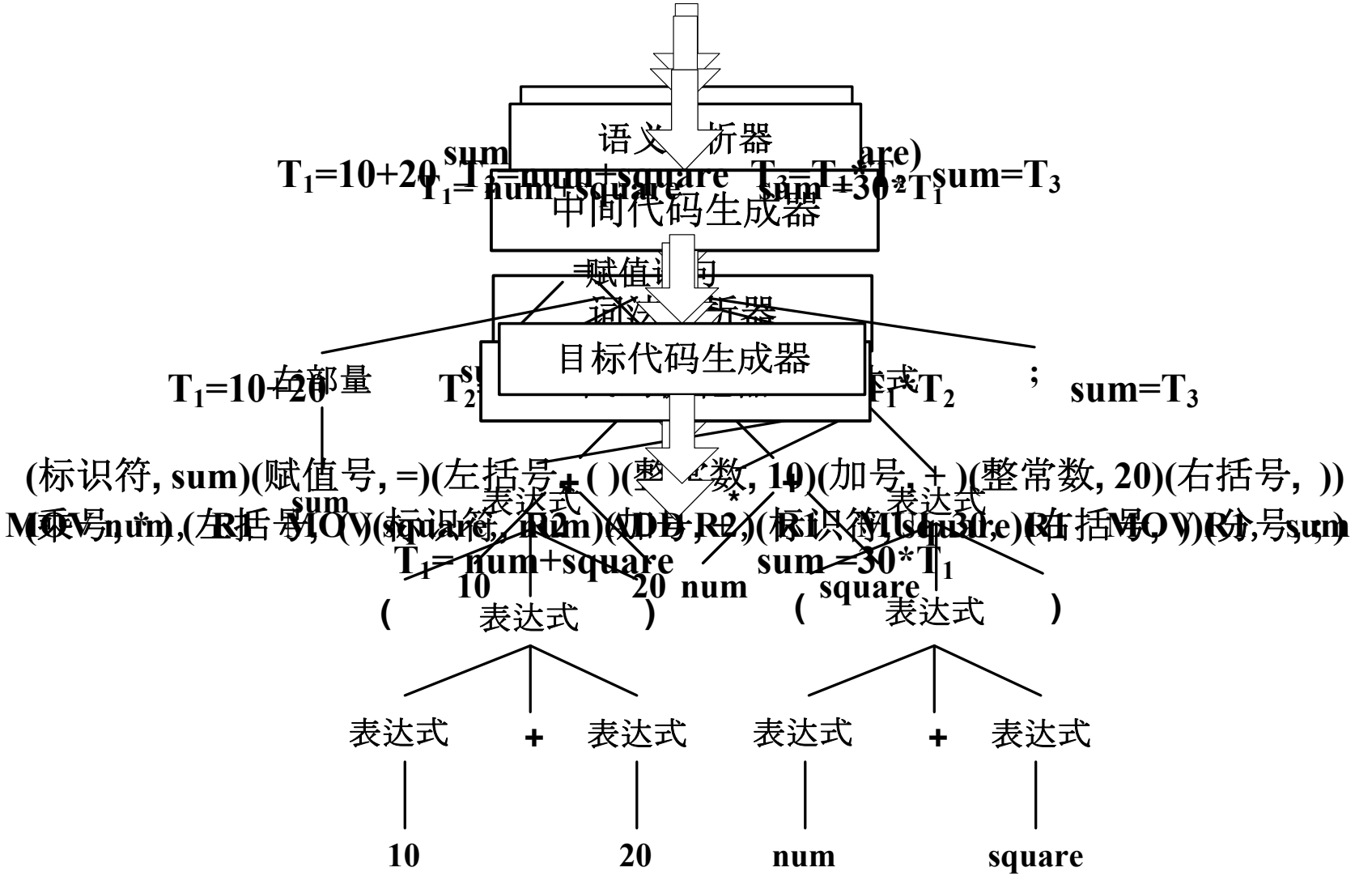
(8) 错误处理

- 进行各种错误的检查、报告、纠正，以及相应的续编译处理(如：错误的定位与局部化)
  - •词法：拼写……
  - •语法：语句结构、表达式结构……
  - •语义：类型不匹配、参数不匹配……

模块分类

- 分析：词法分析、语法分析、语义分析
- 综合：中间代码生成、代码优化、目标代码生成
- 辅助：符号表管理、出错处理
- 8项功能对应8个模块

语句sum=(10+20)*(num+square);的翻译过程



1.4 编译程序的组织

- 根据系统资源的状况、运行目标的要求……等，可以将一个编译程序设计成多遍（Pass）扫描的形式，在每一遍扫描中，完成不同的任务。
  - 如：首遍构造语法树

    二遍处理中间表示、增加信息等。

- 遍可以和阶段相对应，也可以和阶段无关

- 单遍代码不太有效

1.4 编译程序的组织

编译程序的设计目标

- 规模小、速度快、诊断能力强、可靠性高、可移植性好、可扩充性好
- 目标程序也要规模小、执行速度快

编译系统规模较大，因此可移植性很重要

- 为了提高可移植性，将编译程序划分为前端和后端

1.4 编译程序的组织

前端

- 与源语言有关、与目标机无关的部分
- 词法分析、语法分析、语义分析与中间代码生成、与机器无关的代码优化

后端

- 与目标机有关的部分
- 与机器有关的代码优化、目标代码生成

1.5 编译程序的生成

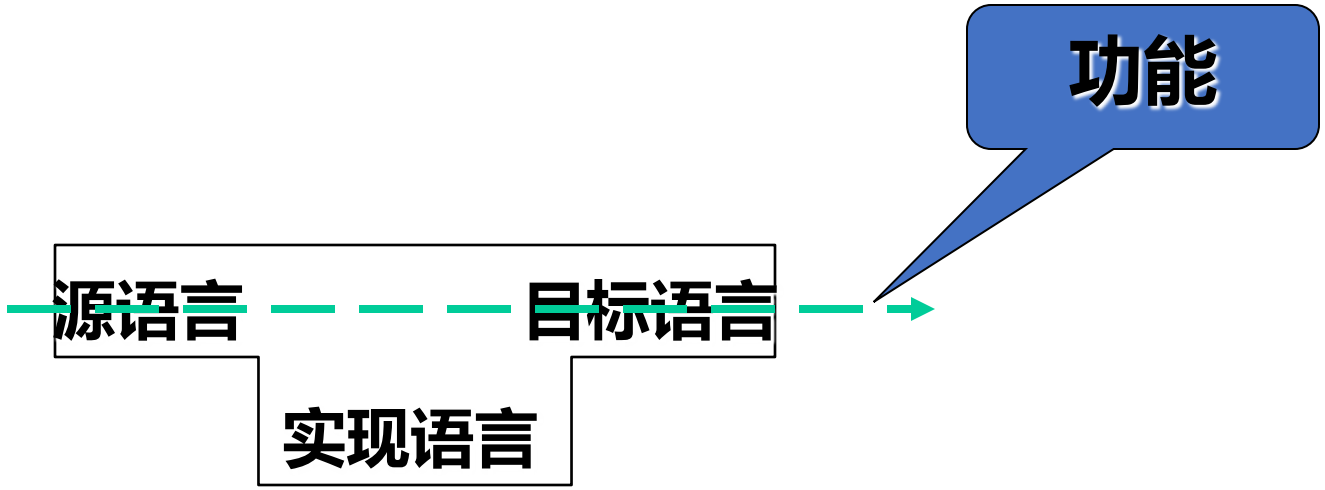
◘如何实现编译器?

- 直接用可运行的代码编制——太费力!
- 自展-使用语言提供的功能来编译该语言自身。
- “第一个编译器是怎样被编译的?”

$$C_1 \subseteq C_2 \subseteq C_3 \subseteq \ldots \subseteq C_n = C\text{语言集合}$$

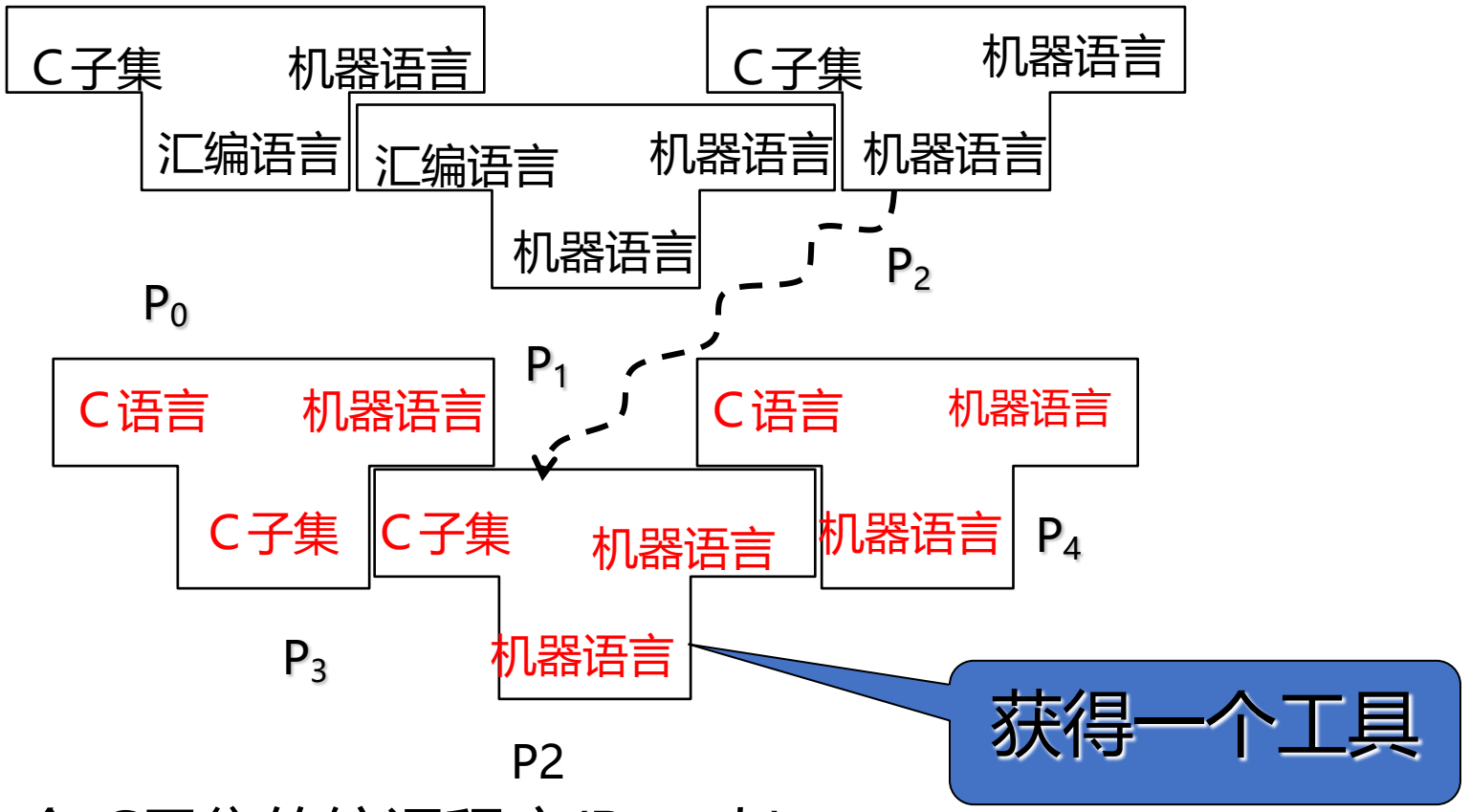
（1）T形图

- 表示语言翻译的 T 形图



（2）自展

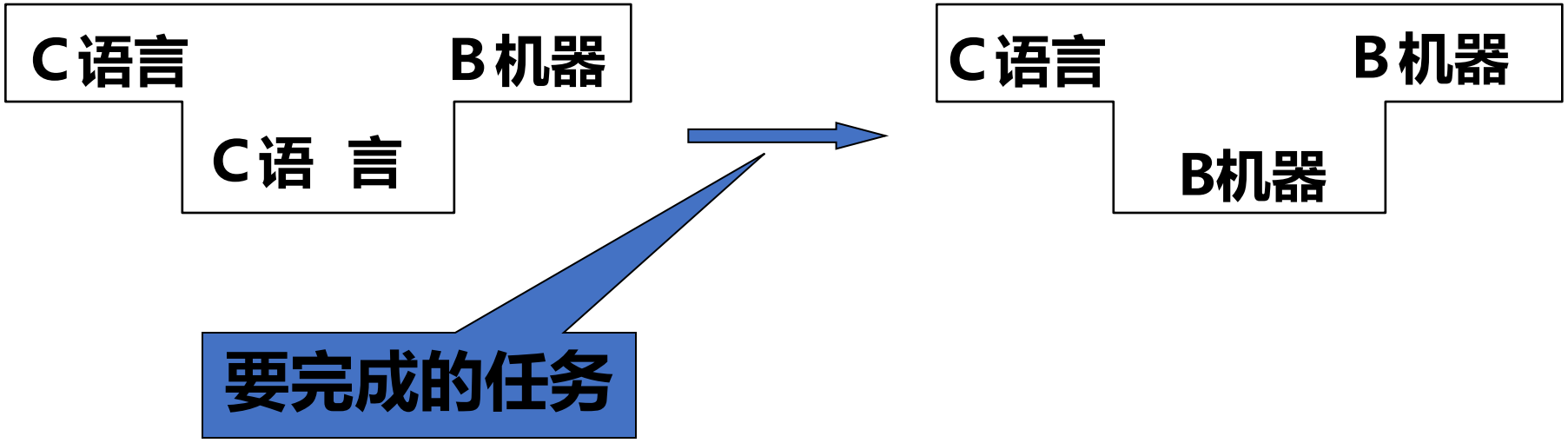
- 问题一：如何直接在一个机器上实现C语言编译器?
- 解决:
  - 用汇编语言实现一个C子集的编译程序(P_0—人)
  - 用汇编程序处理该程序,得到(P_2:可直接运行)
  - 用C子集编制C语言的编译程序(P_3—人)
  - 用P_2编译P_3，得到P_4



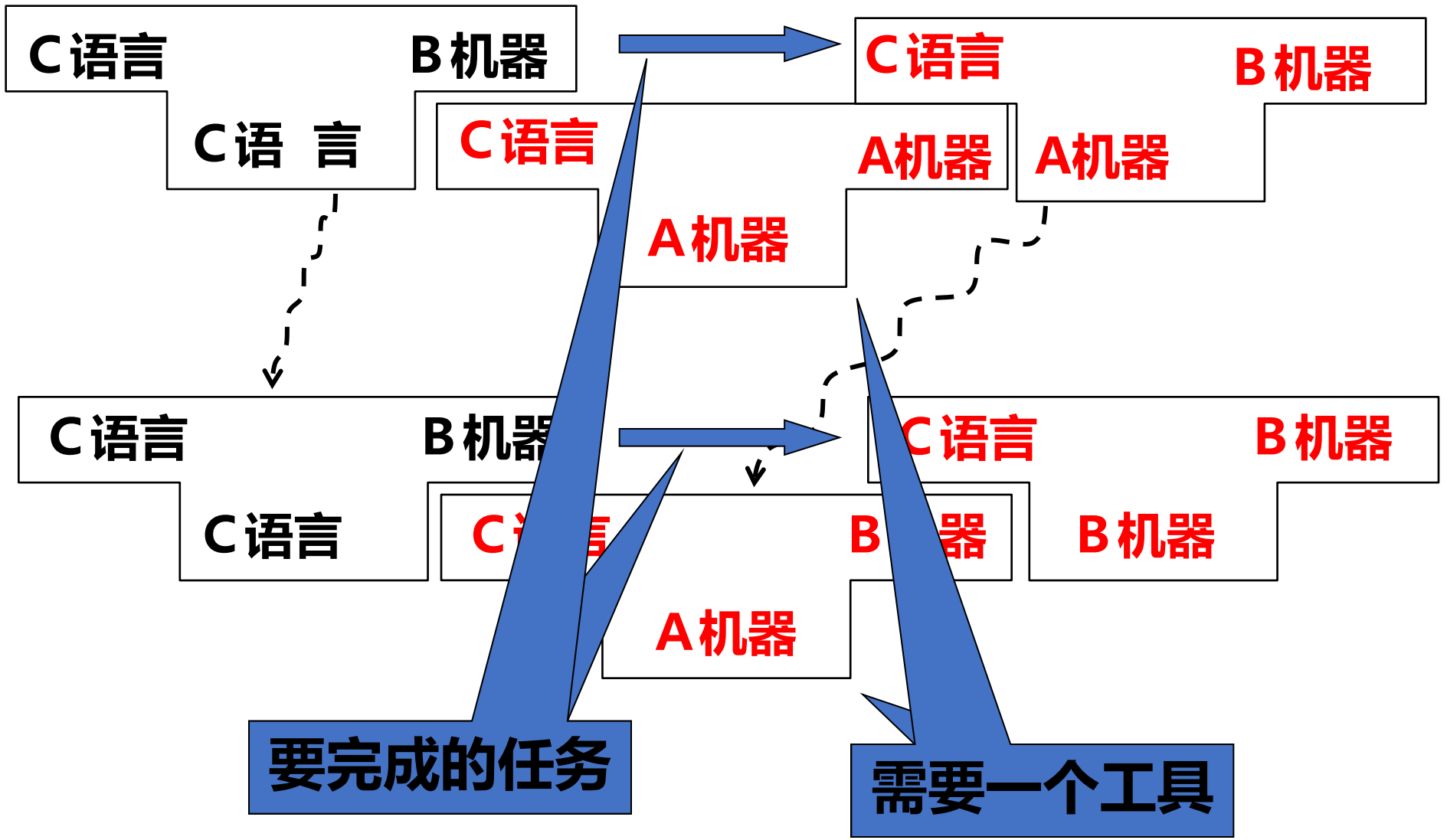
1. 用汇编语言实现一个 C子集的编译程序(P_0—人)
2. 用汇编程序(P_1)处理该程序,得到(P_2:可直接运行)
3. 用C子集编制 C语言的编译程序(P_3—人)
4. 用P_2编译P_3，得到P_4

(3) 移植

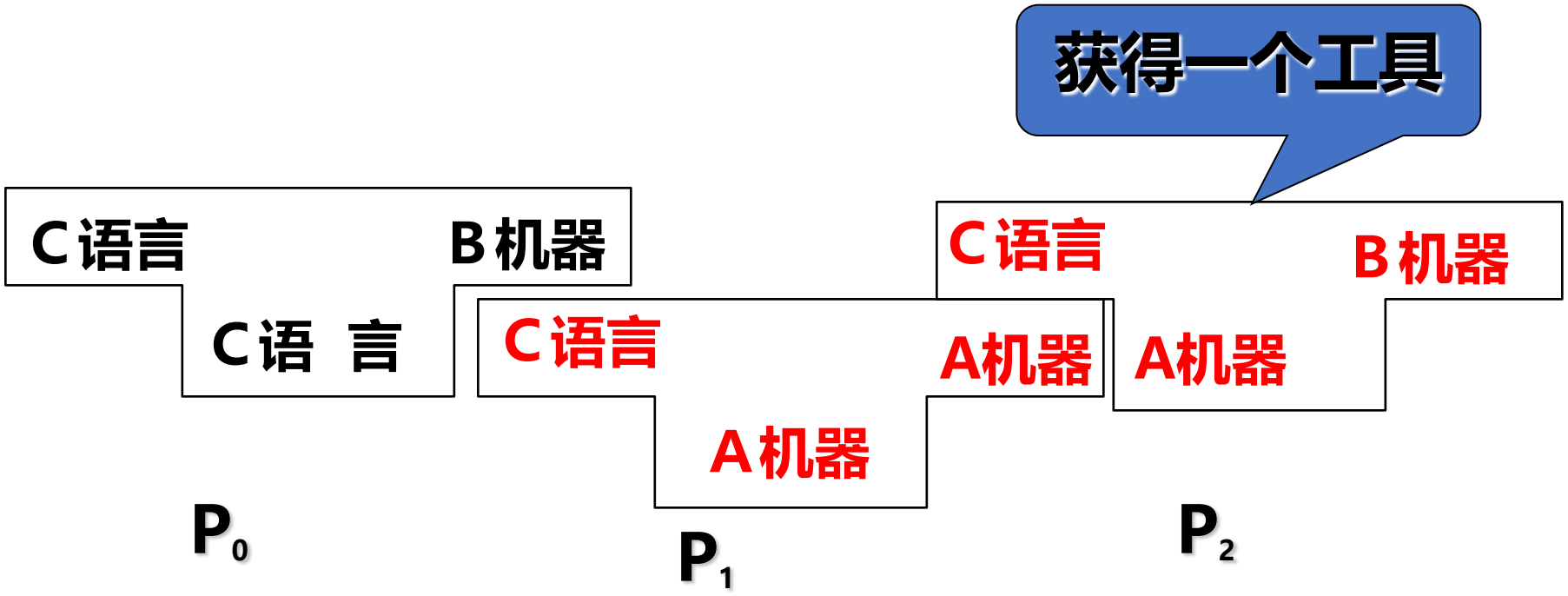
- 问题二：A机上有一个C语言编译器，是否可利用此编译器实现B机上的C语言编译器?
  - 条件：A机有C 语言的编译程序
  - 目的：实现B机的C语言的编译



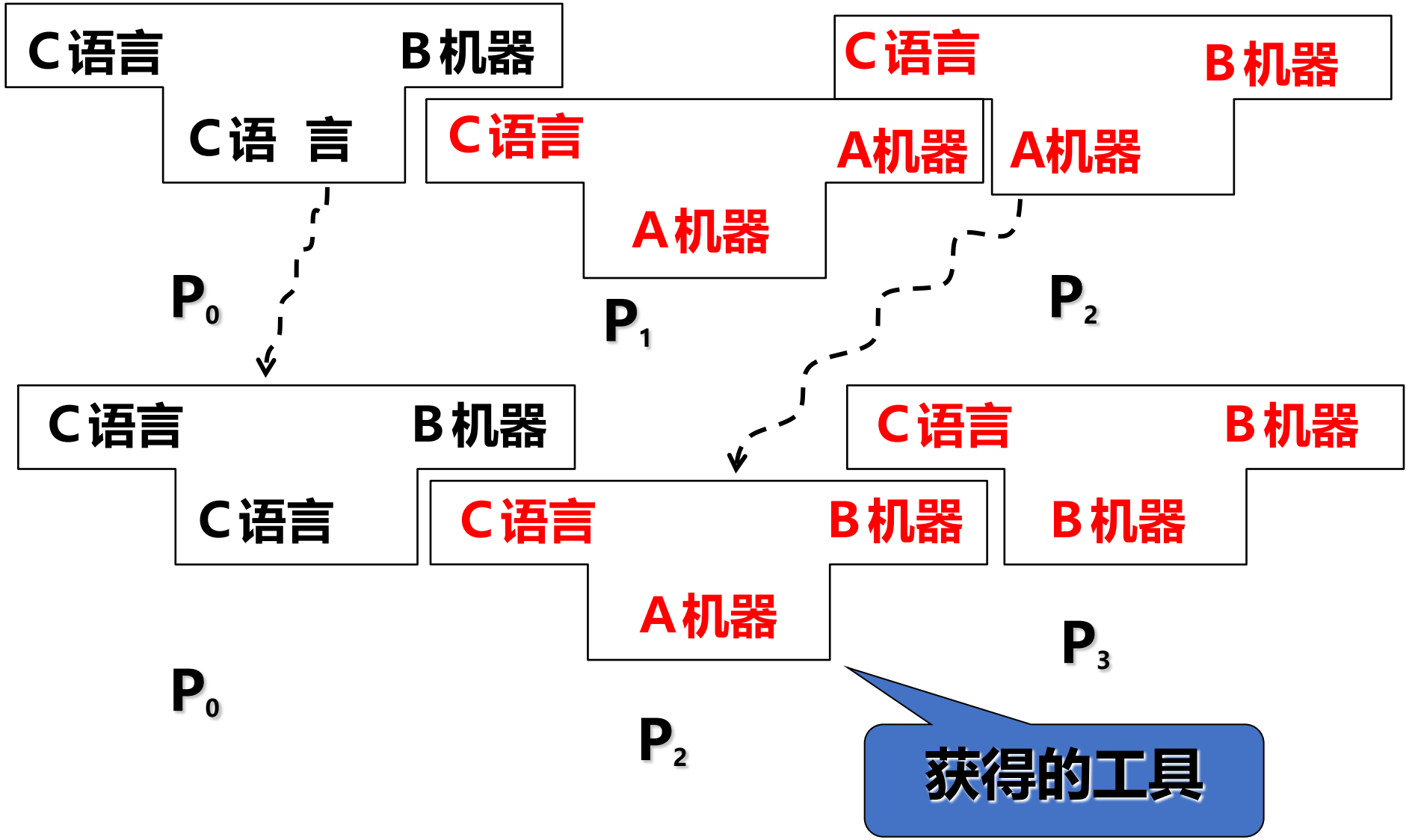
a) 问题的分析


C语言
B机器
C语 言
C语言
A机器
A机器
A机器
C语言
B机器
C语言
C语言
B机器
B机器
A机器
要完成的任务
需要一个工具

b) 问题的解决办法



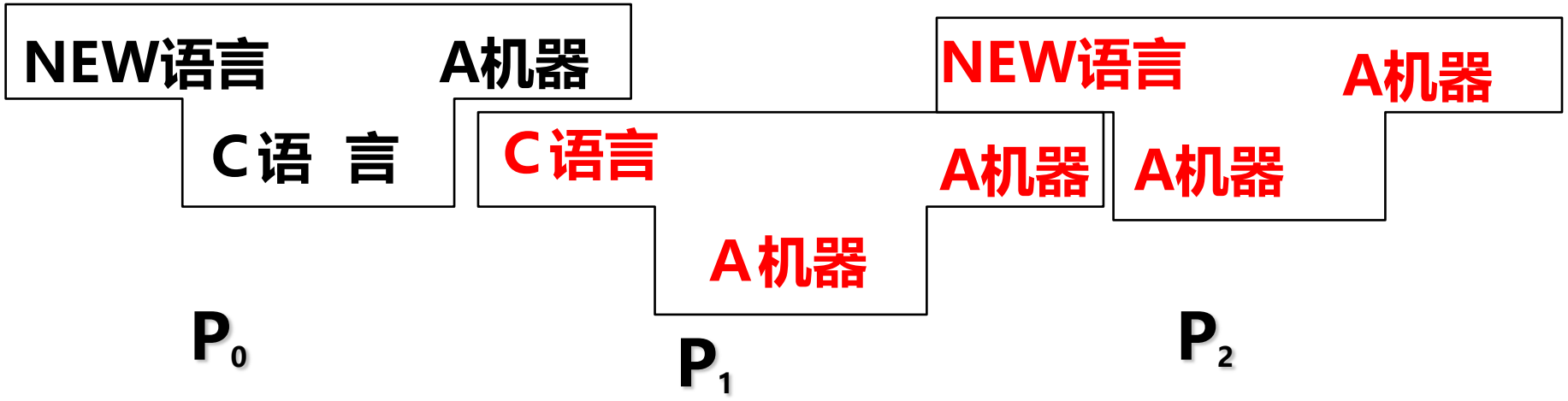
1. (人)用C语言编制B机的C编译程序P_0(C→B)
2. (A机的C编译P_1)编译P_0，得到在A机上可运行的P_2(C→B)



3. (A机的P_2)编译P_0，得到在B机上可运行的P_3(C→B)

（4）本机编译器的利用

- 问题三： A机上有一个C语言编译器，现要实现一个新语言NEW的编译器？能利用交叉编译技术么？
- 用C编写NEW的编译，并用C编译器编译它



多选题 100分

设置

设A机器上有语言L的编译程序，可以用它来编制B机器上的语言L’的编译程序。其中，需要我们另外编写的编译程序为：

A L语言实现的L’ 到A机器语言的编译程序

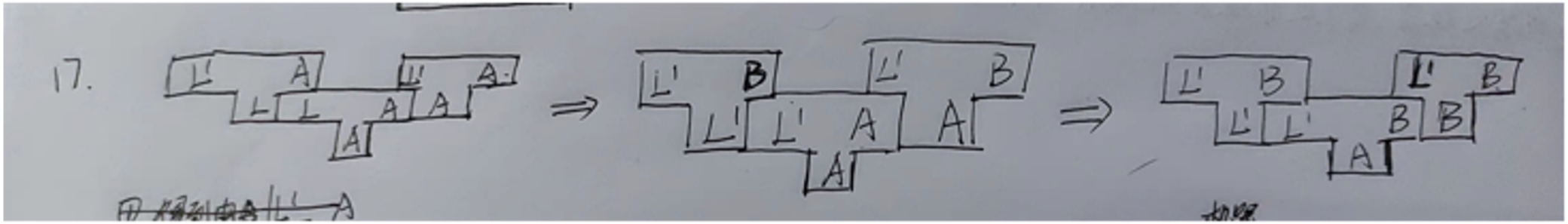
B L’ 语言实现的L’ 到B机器语言的编译程序

C A机器语言实现的L’ 到B机器语言的编译程序

D B机器语言实现的L’ 到B机器语言的编译程序

提交

设A机器上有语言L的编译程序，可以用它来编制B机器上的语言L'的编译程序。其中，需要我们另外编写的编译程序为：



（5） 编译程序的自动生成

词法分析器的自动生成程序



输入：

词法（正规表达式）

识别动作（C程序段）

输出：

yylex( ) 函数

（5） 编译程序的自动生成

- 语法分析器的自动生成程序



输入：

语法规则（产生式）

语义动作（C程序段）

输出：

yyparse( ) 函数

编译技术的应用

把复杂数据看作一条语句

- 数据格式的分析
  - 利用词法分析、语法分析方法

- 数据处理的框架
  - 基于语法制导的语义处理框架

编译技术可以用于各种复杂数据的分析处理

编译技术的应用

- 自然语言的理解和翻译
  - 句子翻译
  - 输入法
  - 语音合成、翻译……
  - 内容过滤
- 语法制导的结构化编辑器
- 程序格式化工具
- 软件测试工具
- 程序理解工具
- 高级语言的翻译工具
- ……

例1-1

- DOS 命令 date 的输出格式
  - 例：9-3-1993、09-03-1993、9-03-93
- 语法
  - date → month - day - year
- 词法
  - month → DIGIT DIGIT | DIGIT
  - day → DIGIT DIGIT | DIGIT
  - year → DIGIT DIGIT | DIGII DIGIT DIGIT DIGIT

例1-1（续）

- 语义
  - year(年)、month(月)、day(日)
- 语义约束条件
  - 0 < month.value < 13
  - 0 < day.value < 32，31，30
  - 0 < year.value < 10000

1.6 本章小结

- 编译原理是一门非常好的课
- 程序设计语言及其发展
- 程序设计语言的翻译
- 编译程序的总体结构
- 编译程序的各个阶段
- 编译程序的组织与生成